

# Audit : DELETE /api/restaurants/{id}/ → 500 Internal Server Error

**Contexte :** La suppression d'un restaurant depuis la page Établissements du frontend renvoie un 500.  
**Endpoint :** DELETE `http://localhost:5174/api/restaurants/34/` → proxy → `http://localhost:8000/api/restaurants/34/` **Date :** 2026-05-16 **Commit backend audité :** `570f074` ("ajout test impression + catégorie fournisseurs") **Statut :** ❌ **Toujours reproductible au dernier pull**

## Le frontend est-il en cause ?

**NON.** Audit complet du flux frontend :

| Étape          | Code                                                                                       | Verdict               |
|----------------|--------------------------------------------------------------------------------------------|-----------------------|
| Hook mutation  | <code>useDeleteEstablishment() → apiDelete(\restaurants/\${id}/)`</code>                   | ✅ Correct             |
| ID envoyé      | <code>deleteTarget.id = String(r.restaurantId) = "34"</code>                               | ✅ Numérique valide    |
| URL construite | <code>http://localhost:5174/api/restaurants/34/</code> (trailing slash OK)                 | ✅ Conforme au swagger |
| Proxy Vite     | <code>/api → http://localhost:8000</code>                                                  | ✅ Correct             |
| Auth header    | Bearer token injecté automatiquement par interceptor                                       | ✅ Correct             |
| CSRF           | X-CSRFToken injecté pour DELETE (non-GET)                                                  | ✅ Correct             |
| Méthode HTTP   | DELETE (correspond à <code>lookup_field = 'id_restaurant'</code> du <code>ViewSet</code> ) | ✅ Correct             |
| Error handling | <code>useMutationWithDefaults → toast "Erreur lors de la suppression"</code>               | ✅ Correct             |

**Le 500 est retourné par le backend Django.** Le frontend ne peut pas causer un 500 — il ne fait qu'envoyer `DELETE /api/restaurants/34/` avec un token valide.

## Ce qui a été corrigé dans les commits récents

Les commits `600173c` → `570f074` ont apporté des améliorations partielles :

| Migration                    | Changement                                                    | Statut                   |
|------------------------------|---------------------------------------------------------------|--------------------------|
| <code>commandes/0006</code>  | <code>CommandeHistoric.restaurant : PROTECT → SET_NULL</code> | ✅ Corrigé dans le modèle |
| <code>menu/0005</code>       | <code>Article.restaurant : CASCADE → SET_NULL</code>          | ✅ Corrigé                |
| <code>restaurant/0007</code> | Création du modèle <code>RestaurantDeleted</code> (archivage) | ✅ En place               |

Mais ces corrections ne suffisent pas. Le `destroy()` n'a toujours pas de gestion d'erreur, et 11 modèles cascaden toujours en dur.

## Cause racine : `destroy()` sans protection (TOUJOURS PRÉSENT)

Le `destroy()` dans `RestaurantViewSet` (`apps/restaurant/views.py`, lignes 64-97) :

```
@transaction.atomic
def destroy(self, request, *args, **kwargs):
    instance = self.get_object()
    archive = RestaurantDeleted.objects.create(...)
    CommandeHistoric.objects.filter(restaurant=instance).update(
        restaurant_deleted=archive,
        restaurant=None,
    )
    return super().destroy(request, *args, **kwargs) # ← TRIGGER CASCADE, PAS DE TRY/EXCEPT
```

### Problèmes :

1. Pas de `try/except` — toute erreur DB (timeout, lock, FK non migrée) → 500 brut
2. Pas de pré-validation — on ne vérifie pas si des commandes actives ou factures bloquent
3. CASCADE massif sur 11 modèles sans avertissement

## Risque migration : PROTECT → SET\_NULL en MySQL

La migration `commandes/0006` change `CommandeHistoric.restaurant` de `PROTECT` à `SET_NULL`.

| Contexte     | Risque                                                                                                                                                                                     |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQLite (dev) | Les FK constraints sont souvent désactivées → la migration passe silencieusement sans recréer la contrainte                                                                                |
| MySQL (prod) | La migration <code>AlterField</code> doit modifier la contrainte FK au niveau DB — si ça n'a pas été fait (migration faked, timeout, erreur silencieuse), la DB garde <code>PROTECT</code> |

### Vérification obligatoire

```
# 1. Vérifier que toutes les migrations sont appliquées
python manage.py showmigrations commandes
python manage.py showmigrations menu
python manage.py showmigrations restaurant

# 2. Vérifier la contrainte DB réelle
python manage.py dbshell

# MySQL :
SHOW CREATE TABLE T_HOLLY_PI_COMMANDES_HISTORIC;
# Chercher la ligne CONSTRAINT ... FOREIGN KEY (`restaurant_id`) ... ON DELETE ...
# Doit être ON DELETE SET NULL (pas ON DELETE RESTRICT/NO ACTION)
```

```
# SQLite :  
.schema T_HOLLY_PI_COMMANDES_HISTORIC
```

## Toutes les FK pointant vers Restaurant (état au commit 570f074)

| App       | Modèle                      | Champ FK   | on_delete | Danger                                          |
|-----------|-----------------------------|------------|-----------|-------------------------------------------------|
| commandes | <b>Commande</b>             | restaurant | CASCADE   | ✗ Supprime les commandes actives                |
| billing   | <b>Facture</b>              | restaurant | CASCADE   | ✗ Supprime les factures (données financières !) |
| inventory | <b>Stock</b>                | restaurant | CASCADE   | ✗ Supprime tout l'inventaire                    |
| inventory | <b>Reapprovisionnement</b>  | restaurant | CASCADE   | ✗ Supprime l'historique d'appro                 |
| salles    | <b>Salle</b>                | restaurant | CASCADE   | ✗ Cascade → Tables → Réservations               |
| planning  | <b>Shift</b>                | restaurant | CASCADE   | ✗ Supprime le planning                          |
| notes     | <b>Note</b>                 | restaurant | CASCADE   | ✗ Supprime les notes                            |
| reports   | <b>Report</b>               | restaurant | CASCADE   | ✗ Supprime les rapports                         |
| suppliers | <b>CommandeFournisseur</b>  | restaurant | CASCADE   | ✗ Supprime les commandes fournisseur            |
| commandes | <b>ImprimanteReseau</b>     | restaurant | CASCADE   | ✗ Supprime les imprimantes                      |
| settings  | <b>NotificationSettings</b> | restaurant | CASCADE   | ⚠ OneToOne, supprime les settings               |
| settings  | <b>BillingSettings</b>      | restaurant | CASCADE   | ⚠ OneToOne, supprime les settings               |
| staff     | <b>RestaurantEmploye</b>    | restaurant | CASCADE   | ⚠ Supprime les associations (pas les employés)  |
| commandes | <b>CommandeHistoric</b>     | restaurant | SET_NULL  | ✅ Corrigé (migration 0006)                      |
| menu      | <b>Article</b>              | restaurant | SET_NULL  | ✅ Corrigé (migration menu/0005)                 |

## Chaîne de cascade complète

```
Restaurant (id=34) DELETE  
├-> Salle CASCADE  
|   └-> Table CASCADE  
|       └-> Reservation CASCADE (+ M2M allergens, diet_types)
```

```
|> Stock CASCADE (tout l'inventaire)
|> Reapprovisionnement CASCADE (historique appro)
|> Commande CASCADE (commandes actives !)
|   ↳ LigneCommande CASCADE
|> Facture CASCADE (données financières !)
|   ↳ LigneFacture CASCADE
|   ↳ Paiement CASCADE
|> Shift CASCADE (tout le planning)
|> CommandeFournisseur CASCADE
|> Note CASCADE
|> Report CASCADE
|> ImprimanteReseau CASCADE
|> NotificationSettings CASCADE
|> BillingSettings CASCADE
|> RestaurantEmploye CASCADE
```

## Fix immédiat (pour débloquer)

### Option A : Vérifier et appliquer les migrations

```
python manage.py showmigrations
python manage.py migrate
```

Si les migrations sont marquées [X] mais la DB n'a pas changé (MySQL) :

```
# Forcer la re-application de la migration critique
python manage.py migrate commandes 0005 --fake
python manage.py migrate commandes 0006
python manage.py migrate menu 0004 --fake
python manage.py migrate menu 0005
```

### Option B : Ajouter un try/except + pré-validation dans destroy()

```
# apps/restaurant/views.py
from django.db import IntegrityError
from rest_framework import status
from rest_framework.response import Response

@transaction.atomic
def destroy(self, request, *args, **kwargs):
    from apps.commandes.models import Commande, CommandeHistoric
    from apps.billing.models import Facture

    instance = self.get_object()

    # Pré-validation : refuser si données actives
    active_orders = Commande.objects.filter(
        restaurant=instance,
        statut__in=["EN_COURS", "VALIDEE"]
```

```

).count()
if active_orders > 0:
    return Response(
        {"detail": f"Impossible de supprimer : {active_orders} commande(s) en
cours."},
        status=status.HTTP_409_CONFLICT,
    )

try:
    user = (
        request.user
        if getattr(request, "user", None) and request.user.is_authenticated
        else None
    )
    archive = RestaurantDeleted.objects.create(
        original_restaurant_id=instance.id_restaurant,
        nom_restaurant=instance.nom_restaurant,
        adresse_restaurant=instance.adresse_restaurant,
        code_postal=instance.code_postal,
        ville=instance.ville,
        numero_telephone=instance.numero_telephone,
        numero_siret=instance.numero_siret,
        code_naf=instance.code_naf,
        pin_restaurant=instance.pin_restaurant,
        logo_url=instance.logo_url,
        latitude=instance.latitude,
        longitude=instance.longitude,
        deleted_by=user,
    )

    # Rattacher l'historique à l'archive avant suppression
    CommandeHistoric.objects.filter(restaurant=instance).update(
        restaurant_deleted=archive,
        restaurant=None,
    )

    # Rattacher les factures à l'archive (évite perte données financières)
    Facture.objects.filter(restaurant=instance).update(restaurant=None)

    return super().destroy(request, *args, **kwargs)

except IntegrityError as e:
    return Response(
        {"detail": f"Impossible de supprimer : une contrainte base de données
bloque l'opération. ({e})"},
        status=status.HTTP_409_CONFLICT,
    )

```

---

## Recommandations long terme

## 1. Changer les `on_delete` pour les données à conserver

```
# billing/models.py – données financières = JAMAIS supprimer
class Facture(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.SET_NULL, null=True)

# reports/models.py – rapports d'analyse
class Report(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.SET_NULL, null=True)

# suppliers/models.py – historique fournisseurs
class CommandeFournisseur(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.SET_NULL, null=True)

# inventory/models.py – historique appro
class Reapprovisionnement(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.SET_NULL, null=True)
```

## 2. Soft-delete pour Restaurant

```
class Restaurant(models.Model):
    deleted_at = models.DateTimeField(null=True, blank=True)

    def soft_delete(self):
        self.deleted_at = timezone.now()
        self.save(update_fields=["deleted_at"])
```

## 3. PROTECT sur les commandes actives

```
class Commande(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.PROTECT)
    # → Force à clôturer les commandes AVANT de supprimer
```

## Fichiers backend à modifier

| Fichier                  | Action                                                    | Priorité |
|--------------------------|-----------------------------------------------------------|----------|
| apps/restaurant/views.py | Ajouter try/except + pré-validation dans destroy()        | P0       |
| apps/billing/models.py   | Facture.restaurant → SET_NULL + migration                 | P0       |
| apps/reports/models.py   | Report.restaurant → SET_NULL + migration                  | P1       |
| apps/suppliers/models.py | CommandeFournisseur.restaurant → SET_NULL + migration     | P1       |
| apps/inventory/models.py | Reapprovisionnement.restaurant → SET_NULL + migration     | P1       |
| DB MySQL                 | Vérifier que les migrations FK sont réellement appliquées | P0       |

## Étapes de debug

```
# 1. Vérifier les migrations
python manage.py showmigrations

# 2. Tester en shell Django (le traceback exact apparaîtra)
python manage.py shell
>>> from apps.restaurant.models import Restaurant
>>> r = Restaurant.objects.get(id_restaurant=34)
>>> r.delete() # L'erreur exacte s'affiche ici

# 3. Vérifier la contrainte FK réelle en DB
python manage.py dbshell
# MySQL :
SHOW CREATE TABLE T_HOLLY_PI_COMMANDES_HISTORIC;
# SQLite :
.schema T_HOLLY_PI_COMMANDES_HISTORIC

# 4. Logs Django pendant le DELETE (terminal runserver)
# Le traceback complet avec le nom exact de la contrainte violée
```